

Competition

- Distribution shift (added noise to the final test set):
 - Use regularization: models may overfit the train/public test dataset and can help.
 - Cross-validate: Instead of a single split, try k-fold validation.
- Missing values:
 - Impute inside the model: you can try adding the missing values such as median for numeric, most-frequent for categorical.
- Class imbalance:
 - Use class weighted loss such as weighted cross-entropy: this would give more weight for incorrectly predicting the minority class.
 - Oversampling: you can increase the number of minority samples that the model sees during training (using methods such as SMOTE).
 - Tune the decision threshold: instead of defaulting to >0.5 positive class and <0.5 negative class, you can adjust this threshold so the model doesn't ignore the minority class.



UNIVERSITY OF
TORONTO

Engineering

Deep Learning

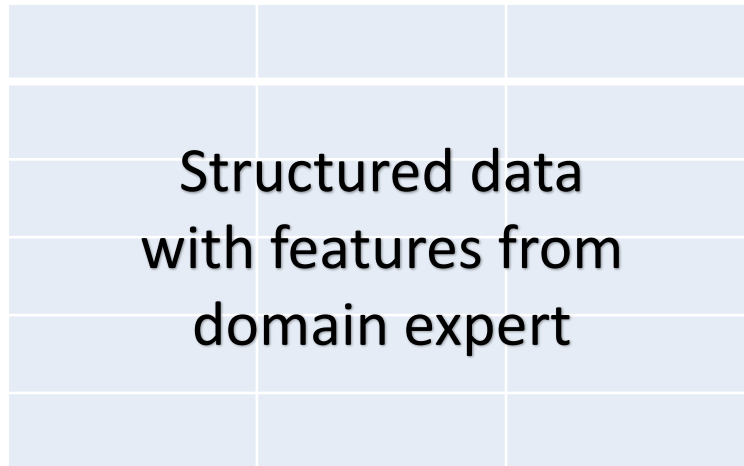
Motivations behind Deep Learning

- ❖ Simple machine learning models struggle on tasks with complex data
- ❖ What are the obstacles faced by ML algorithms?

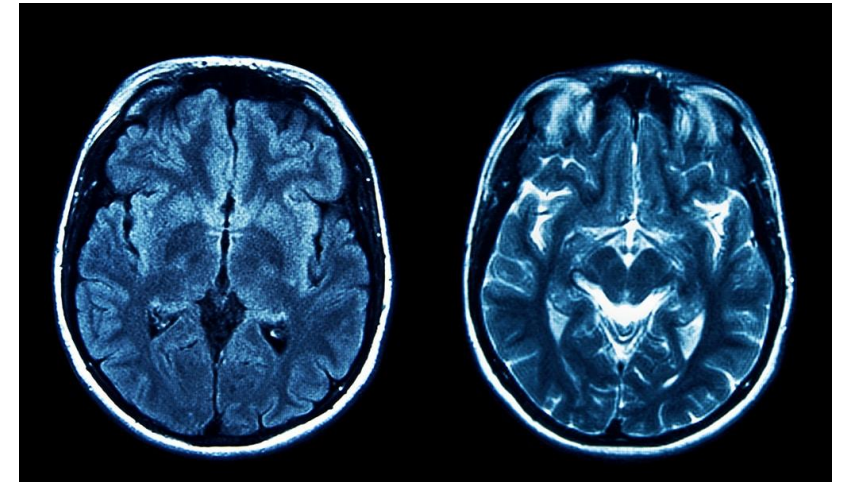
Motivations behind Deep Learning

❖ Representation of the given data.

- Why is it challenging to build valuable features all the time?



v/s



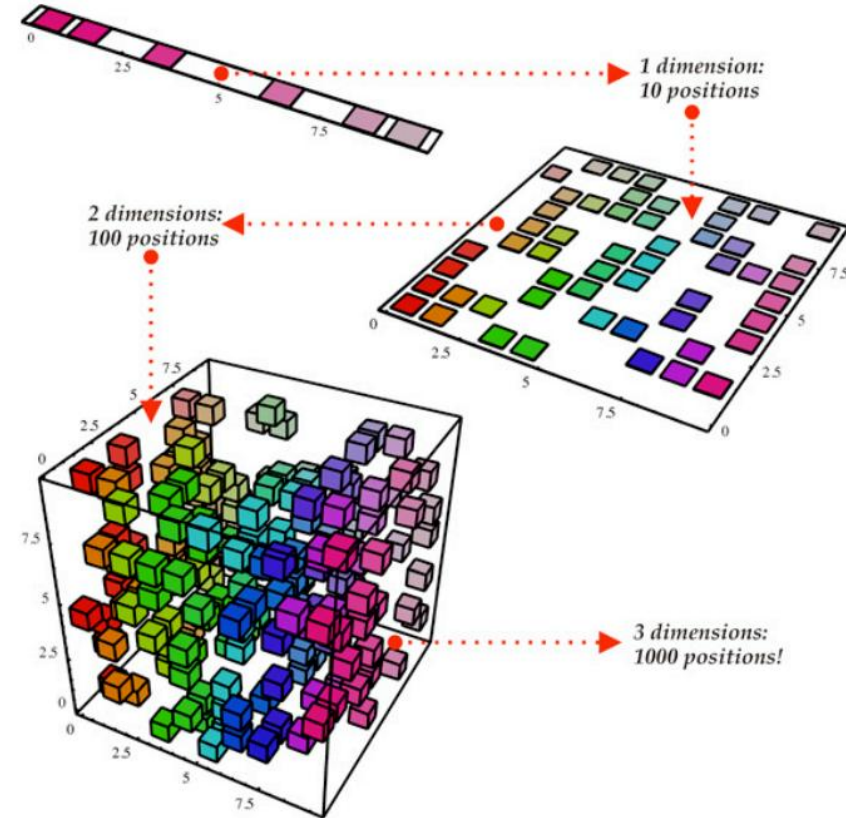
[Image Source](#)

Motivations behind Deep Learning

❖ Number of Dimensions

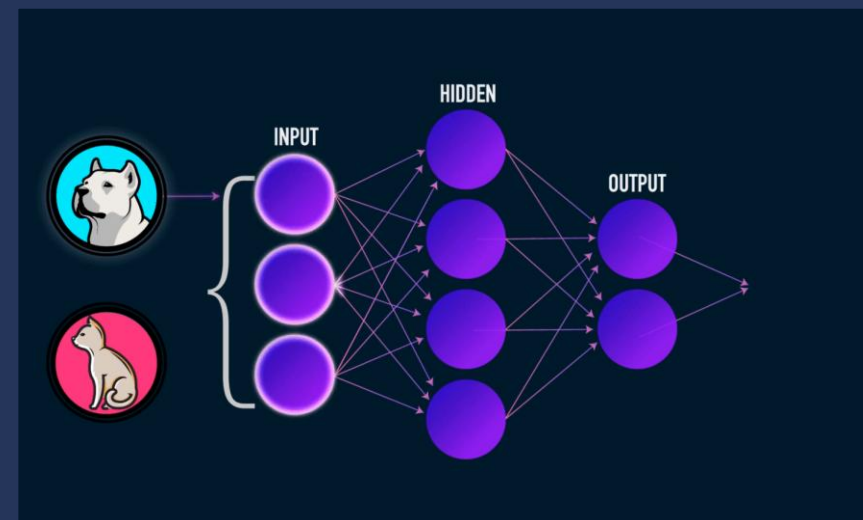
(Curse of Dimensionality)

- Why do you think the performance degrades?

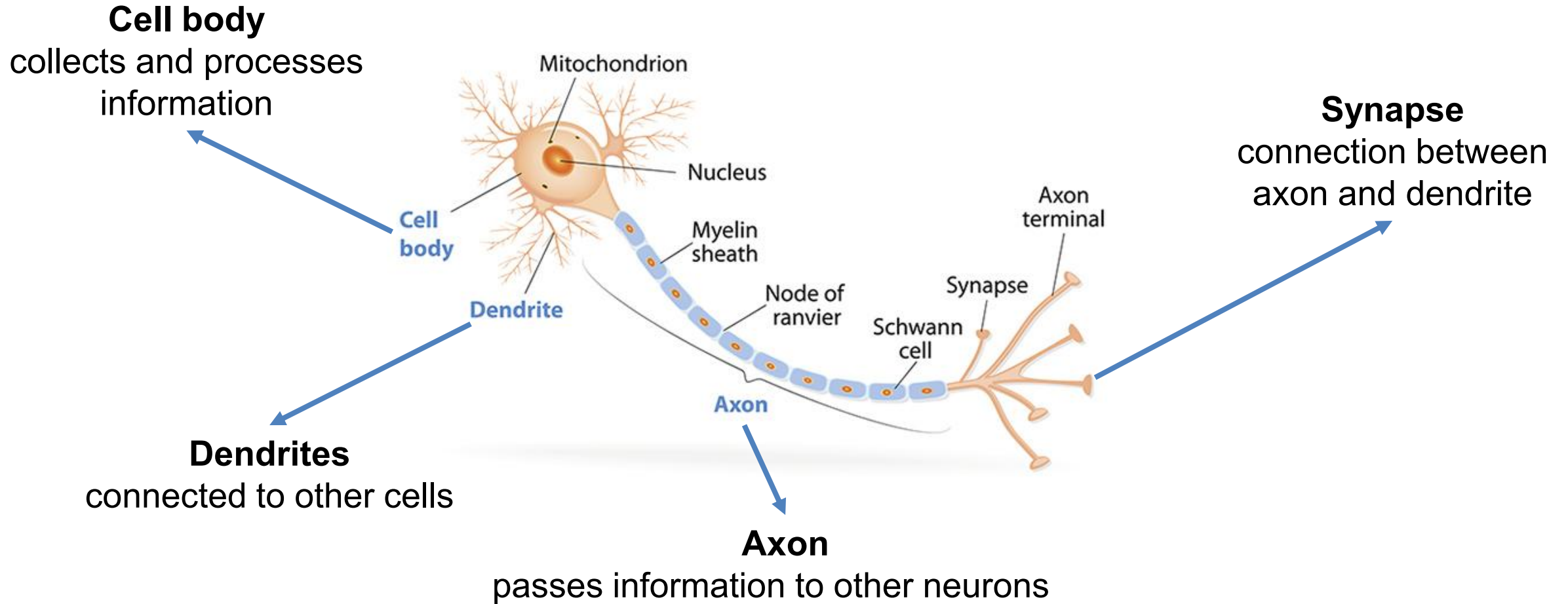


[Source Image: Curse of Dimensionality](#)

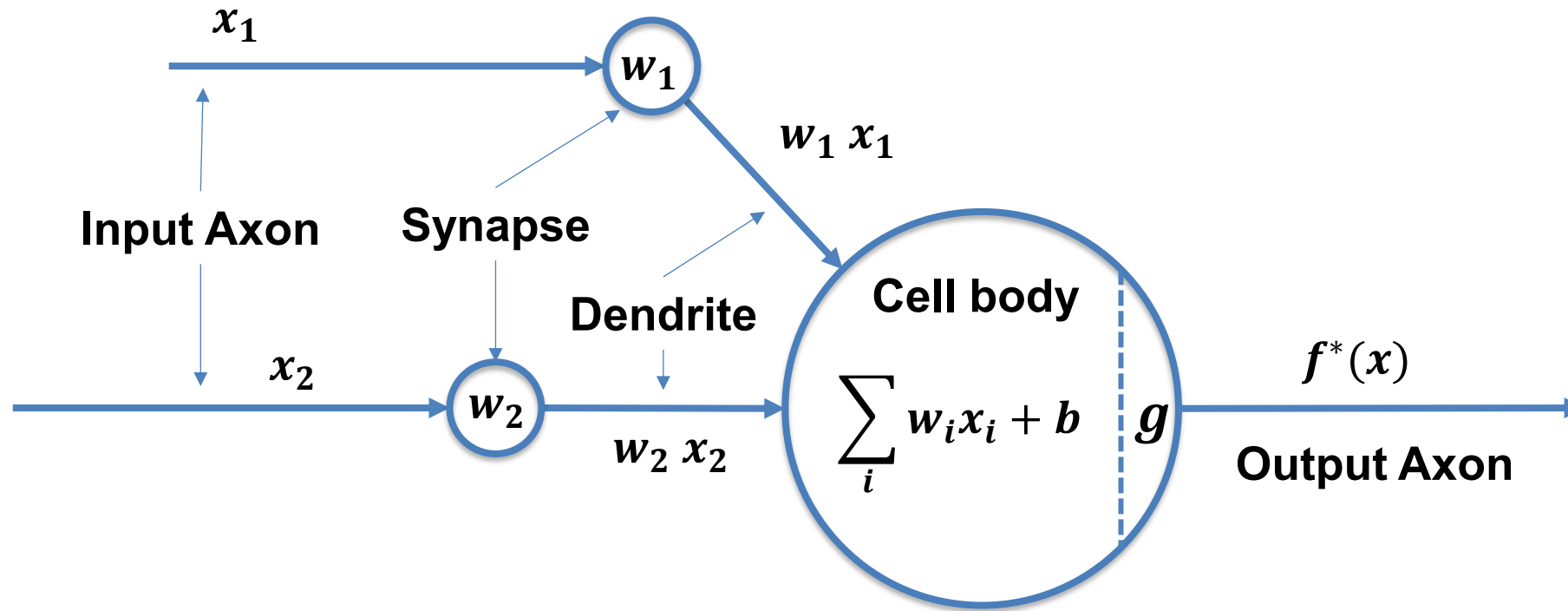
Deep Feedforward Networks



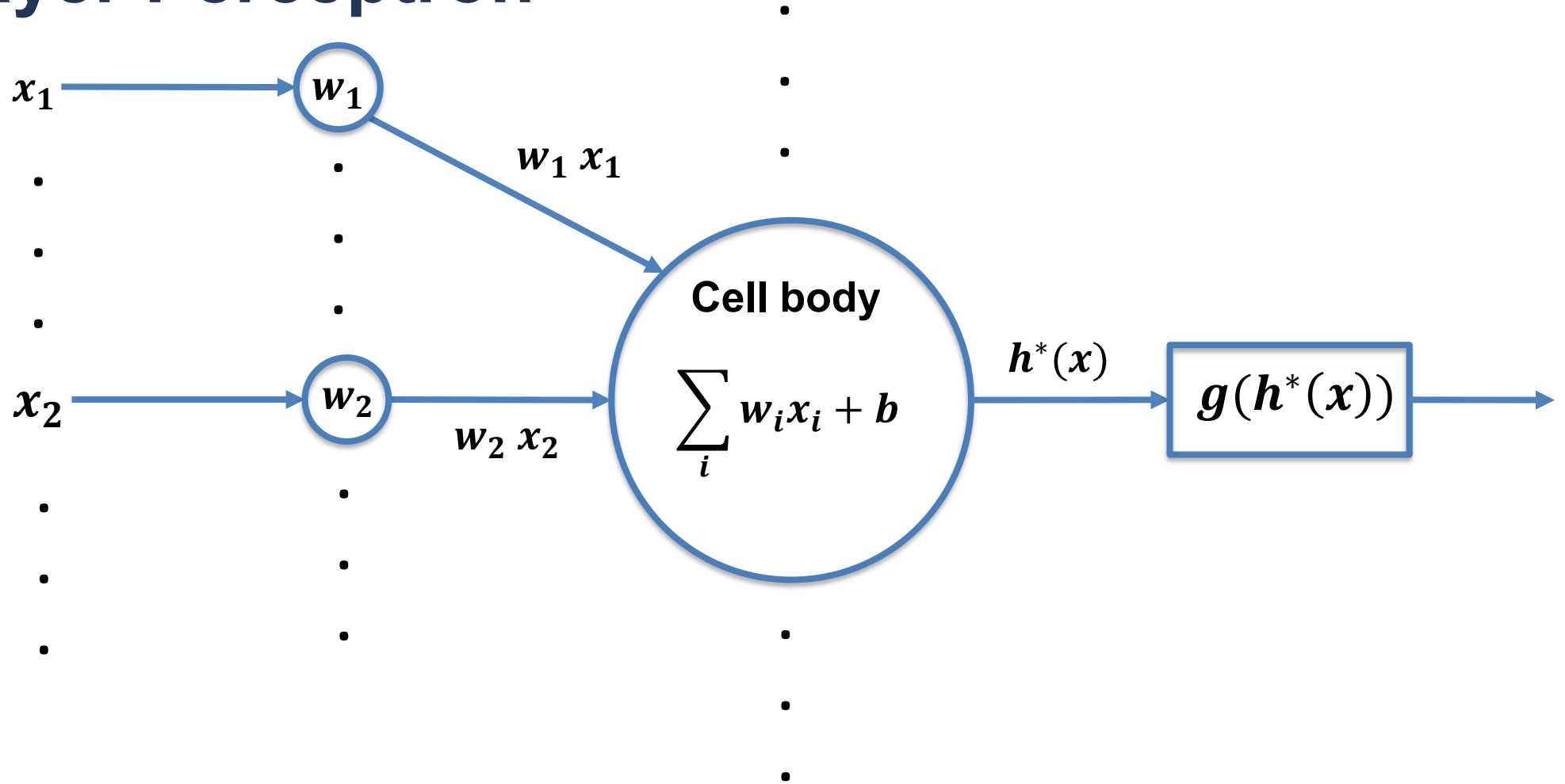
Neural Network Inspiration



Neural Network Inspiration

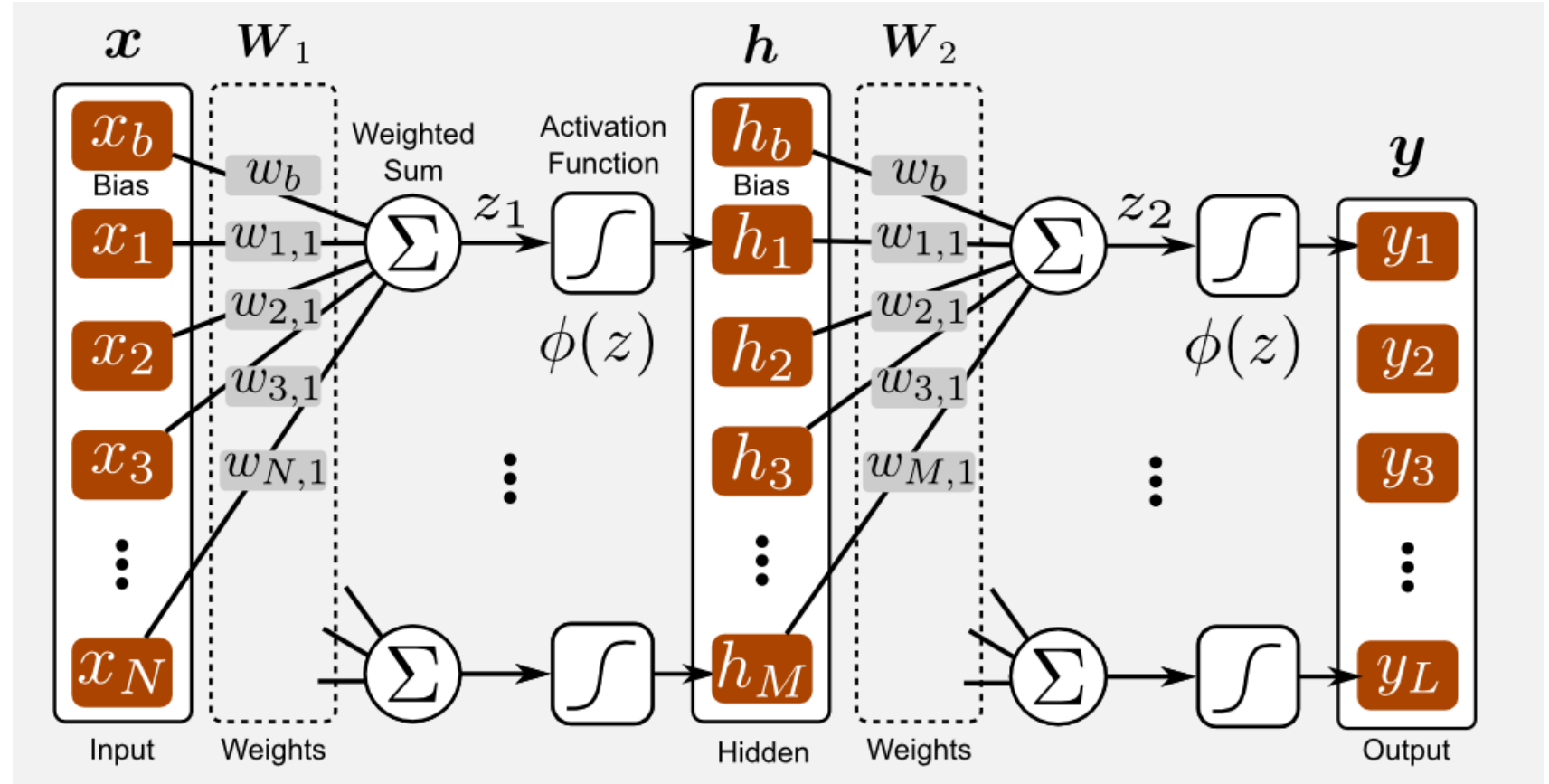


Multilayer Perceptron



Terminology

- ❖ Depth
- ❖ Input Layer
- ❖ Output Layer
- ❖ Hidden Layer
- ❖ Activations



[Image Source](#)

Design Considerations

- ❖ **Cost Functions**
- ❖ **Output Activation Units**
- ❖ **Hidden Activation Units**
- ❖ **Architecture Design**

Cost Functions

❖ Mean Squared Error (**regression**): $\frac{1}{2N} \sum_{n=1}^N ||\widehat{y}_n - y_n||^2 + \text{const}$

❖ Cross-Entropy Loss (**classification**): $-\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K t_{n,k} \log(y_{n,k})$

Note: Mean Squared Error leads to poor results when used with saturating output units. Choice of activation function is critical here.

Activation Functions

$$f_1(f_2 \dots f_n(x))$$

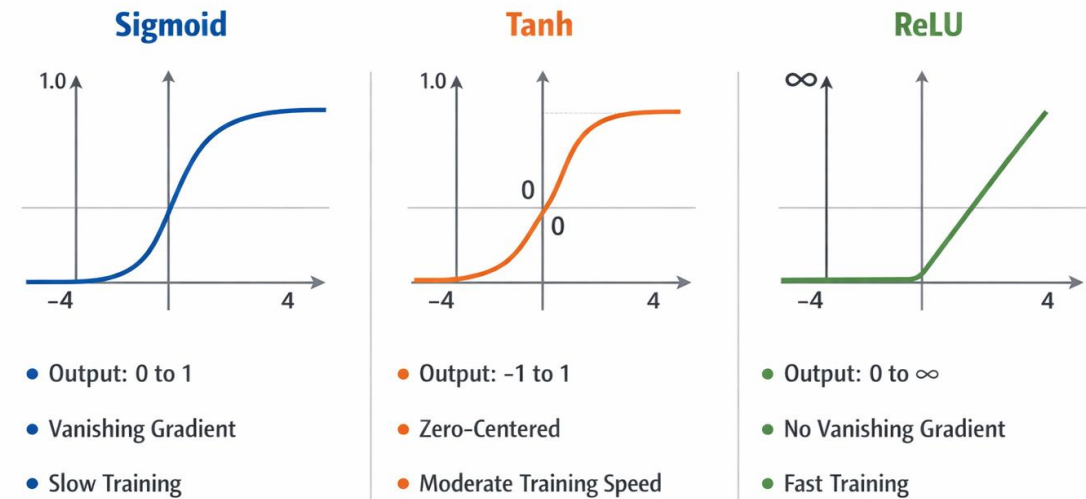
❖ We use activation functions for non-linearity.

❖ Non-linear activation functions:

- Sigmoid function
- tanh function (Hyperbolic Tangent)
- ReLU function (Rectified Linear Unit)

Sigmoid vs Tanh vs ReLU:

Activation Function Comparison



Design choices for Output Units

❖ Linear Units for Gaussian Output Distributions:

- This is a simple output unit with no non-linearity and doesn't create any issue for gradient-based optimization.
- Given h , this output unit produces $\hat{y} = W^T h + b$

Design choices for Output Units

❖ Sigmoid Units for Bernoulli Output Distributions:

- Used when we are predicting binary variable.
- The maximum likelihood approach defines a Bernoulli distribution with just one class, i.e., $P(y = 1|x)$. The output is constrained from $[0, 1]$
- This leads us to a sigmoid output unit.

Design choices for Output Units

❖ Softmax Units for Multinomial Output Distributions:

- Used when we have multiple classes. To obtain the desired output, we exponentiate and normalize z , which is the output of the linear layer.

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Design choices for Hidden Units

- ❖ The choice of activation functions in hidden units is difficult, and trial and error is normally used.

Note: Neural networks do not focus on differentiability of activation functions. They usually return one of the one-sided derivatives.

Design choices for Hidden Units

❖ Rectified Linear Units:

- Easy to optimize as its behavior is close to linear.
- Drawback: They cannot learn when the activation outputs zero.

□ Three generalizations: $\max(0, x) - \alpha \min(0, x)$

- Absolute value rectifications: $\alpha = -1$; $g(z) = |z|$
- Leaky ReLU: $\alpha = 0.01$
- Parametric ReLU: α is *learnable*

Architecture Design

- ❖ **Depth of the network:** Number of Layers
- ❖ **Width of each layer:** Number of units in the layers

Architecture Design

$$f(f(f(f(x))))$$

A **large single-layered** MLP will be able to represent any function, but learning the function can fail:

- Optimization algorithm is unable to find the required parameters.
- Algorithm can choose a wrong function that overfits.

Deeper networks with fewer parameters usually generalize well.

Some Consideration in Architecture Design

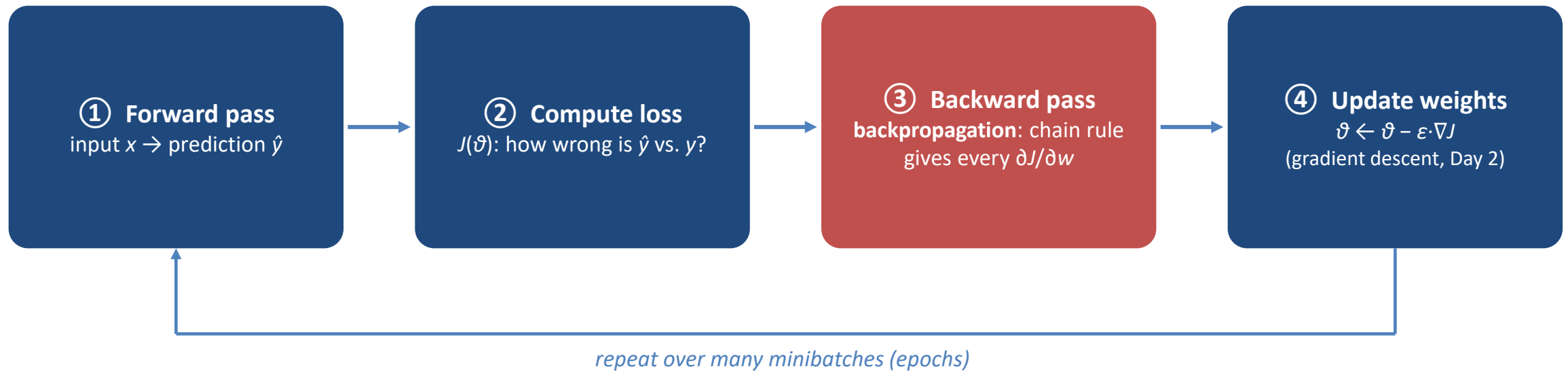
- ❖ **Skip Connections:** helps with gradient flow
- ❖ **Connection between layers:**
 - **MLP** uses a linear transformation with a weight matrix. Every input unit is connected to every output unit.
 - **CNN** uses specialized kernels. We will discuss these in later sections.

How do we train these deep networks?

- ❖ Once the architecture is built, we pass the input x and get an output $\hat{y}$ using **forward-propagation**.
- ❖ Calculate the cost function, $J(\theta)$
- ❖ To optimize, calculate gradients using **back-propagation**.
- ❖ To perform learning, **gradient descent** is used.

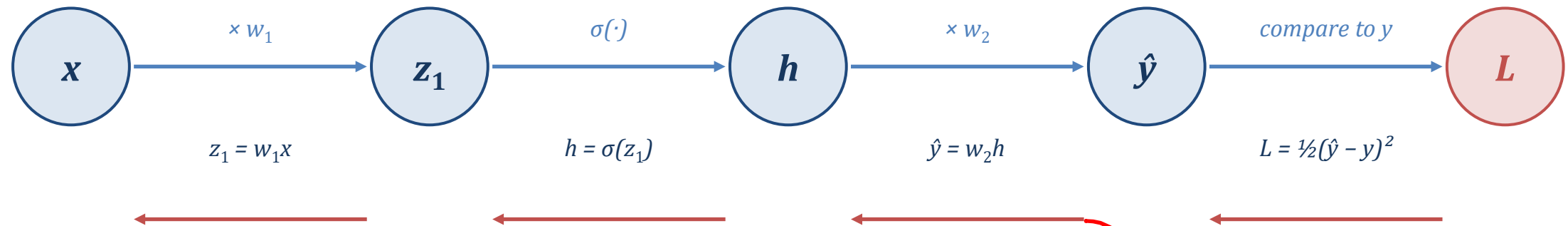
How do we train these deep networks?

- ❖ Same recipe as Day 2 — gradient descent. The only new question: **how do we get the gradients?**



- ❖ Steps ①, ②, ④ you already know — the only new machinery is step ③.
- ❖ **Backpropagation** = the chain rule, organized so one backward sweep yields every gradient.

Backpropagation: the chain rule, layer by layer



backward pass — apply the chain rule from L back to every weight, right to left

$$\partial L / \partial \hat{y} = (\hat{y} - y)$$

$$\partial L / \partial w_2 = (\hat{y} - y) \cdot h$$

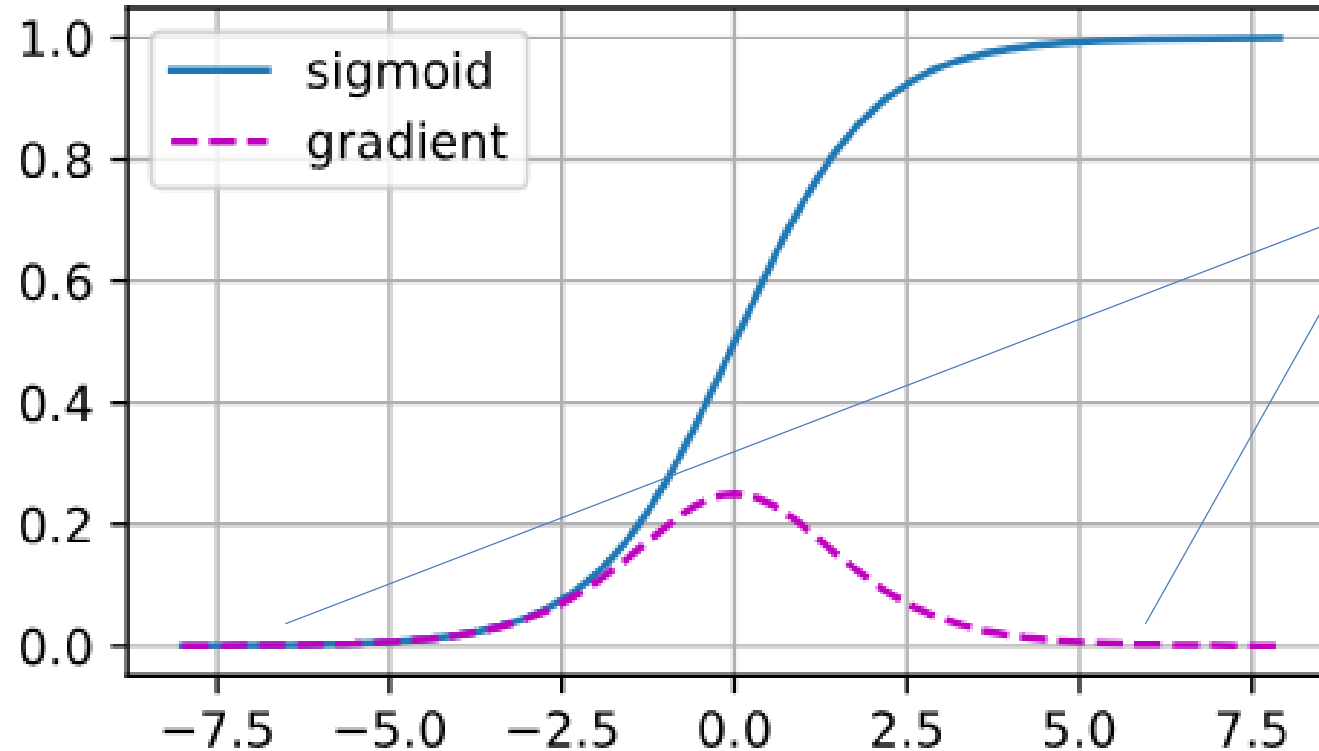
$$\partial L / \partial w_1 = (\hat{y} - y) \cdot w_2 \cdot \sigma'(z_1) \cdot x$$

Handwritten red text: $f_1(f_2(f_3(L)))$

$(\hat{y} - y)$ is computed once at the output, then reused by every layer below — nothing is recomputed.

- ❖ Each extra layer multiplies one more local derivative onto the gradient — that growing product is what backprop computes.

Vanishing and Exploding Gradients



[Image Source](#)

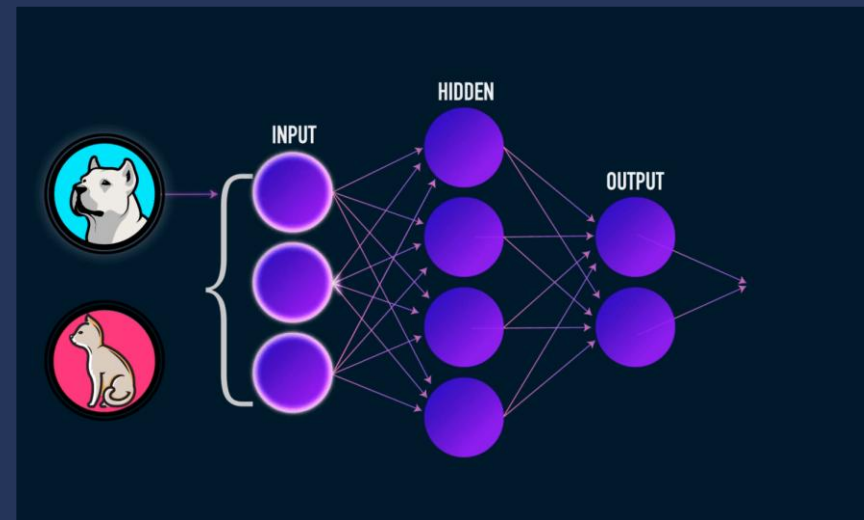
Vanishing Gradients

The gradients on both ends of the sigmoid function are zero. This leads to a vanishing gradient problem.

Exploding Gradients

This is hard to pinpoint. Due to the chain multiplication of 100s of matrices, at times, the gradients become extremely large.

Regularization for Deep Learning



Regularization Strategies

❖ Parameter Norm Penalties

- **Objective Function:** $\tilde{J}(\theta; X, y) = J(\theta; X, y) + \alpha p(\theta)$; where $\alpha \in [0, \infty]$ and p is the penalty term.
- Only weights are penalized, biases are not.
- Sometimes, it is desirable to use a separate penalty term for each layer.

Regularization Strategies: Norm Penalties

□ **L2 Parameter Regularization:** $p(w) = \frac{\alpha}{2} w^T w$

- $\nabla_w \tilde{J}(\theta; X, y) = \alpha w + \nabla_w J(w; X, y)$
- Update: $w \leftarrow w - \epsilon(\alpha w + \nabla_w J(w; X, y)) = (1 - \epsilon\alpha)w - \epsilon \nabla_w J(w; X, y)$
- We can see that the weight vector is shrunk by a constant factor on each step, leading to less complex models.

Regularization Strategies: Norm Penalties

□ L1 Parameter Regularization: $p(w) = \sum |w|$

- $\nabla_w \tilde{J}(\theta; X, y) = \alpha \text{sign}(w) + \nabla_w J(w; X, y)$
- Update: $w \leftarrow w - \epsilon(\alpha \text{sign}(w) + \nabla_w J(w; X, y))$
- Like L2 regularization, L1 also encourages zero coefficients.

Regularization Strategies: Data Augmentation

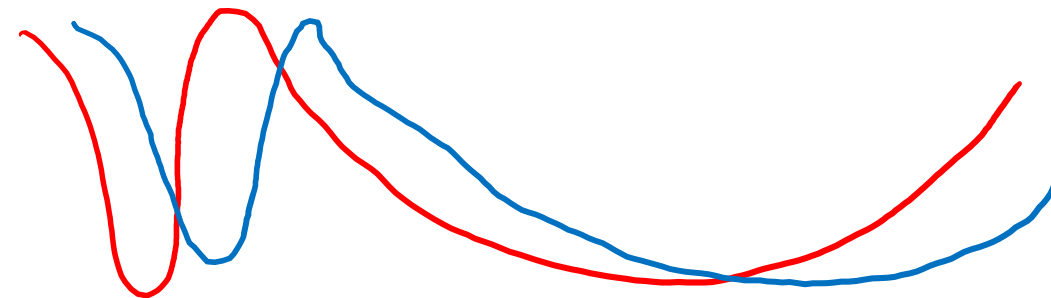
- ❖ More data → a Better generalized model
- ❖ Data is limited. Another way to increase the dataset size is augmentation.

Computer Vision Tasks: Translation, Flips, Random cropping, Rotating, Color

Speech Recognition: Speeding up, Slowing down, Jitter, Pitch shift

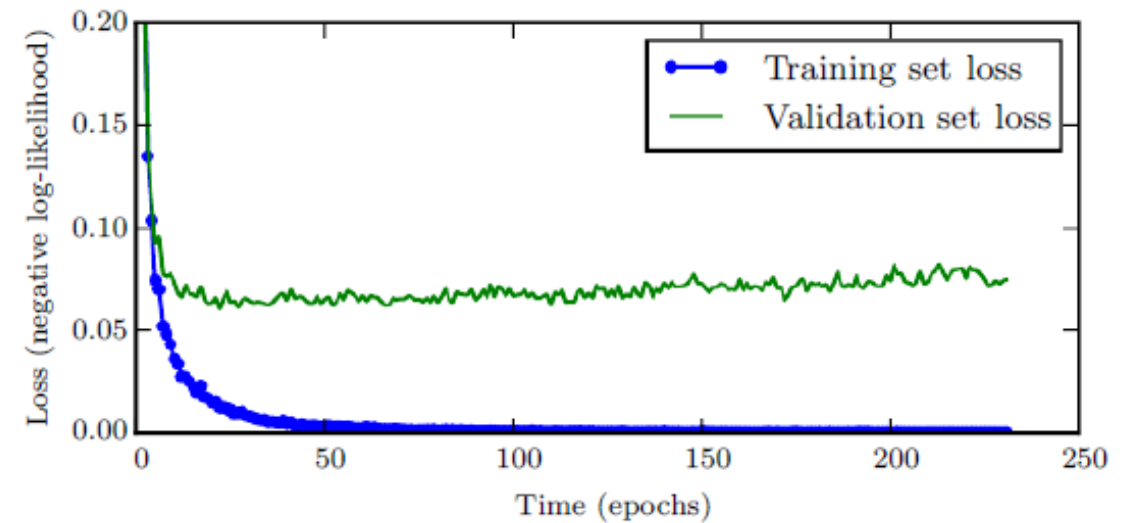
Regularization Strategies: Noise Robustness

- ❖ An interesting way of increasing model robustness is adding noise to network weights.
- ❖ It encourages the parameters to go to regions of parameter space where small perturbations of weights have a small influence on the output → **points of minima surrounded by flat regions.**



Regularization Strategies: Early Stopping

- ❖ Checkpoints of the model are stored at every training epoch.
- ❖ The checkpoint that has the lowest validation error is selected.

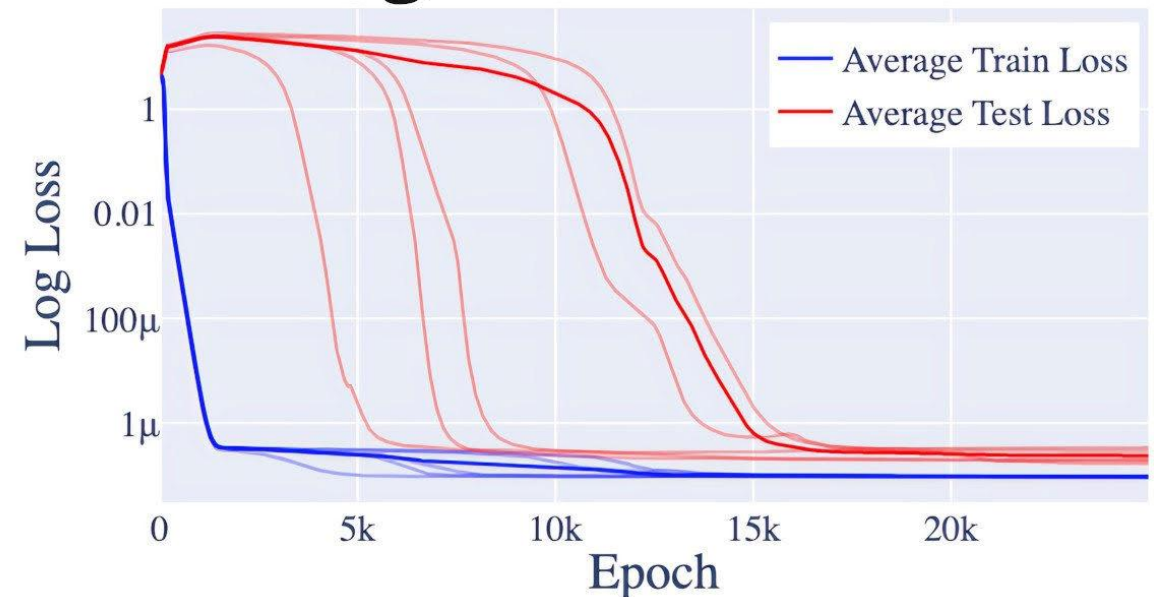


[Image Source](#)

Grokking: a curious exception — in a very specific regime

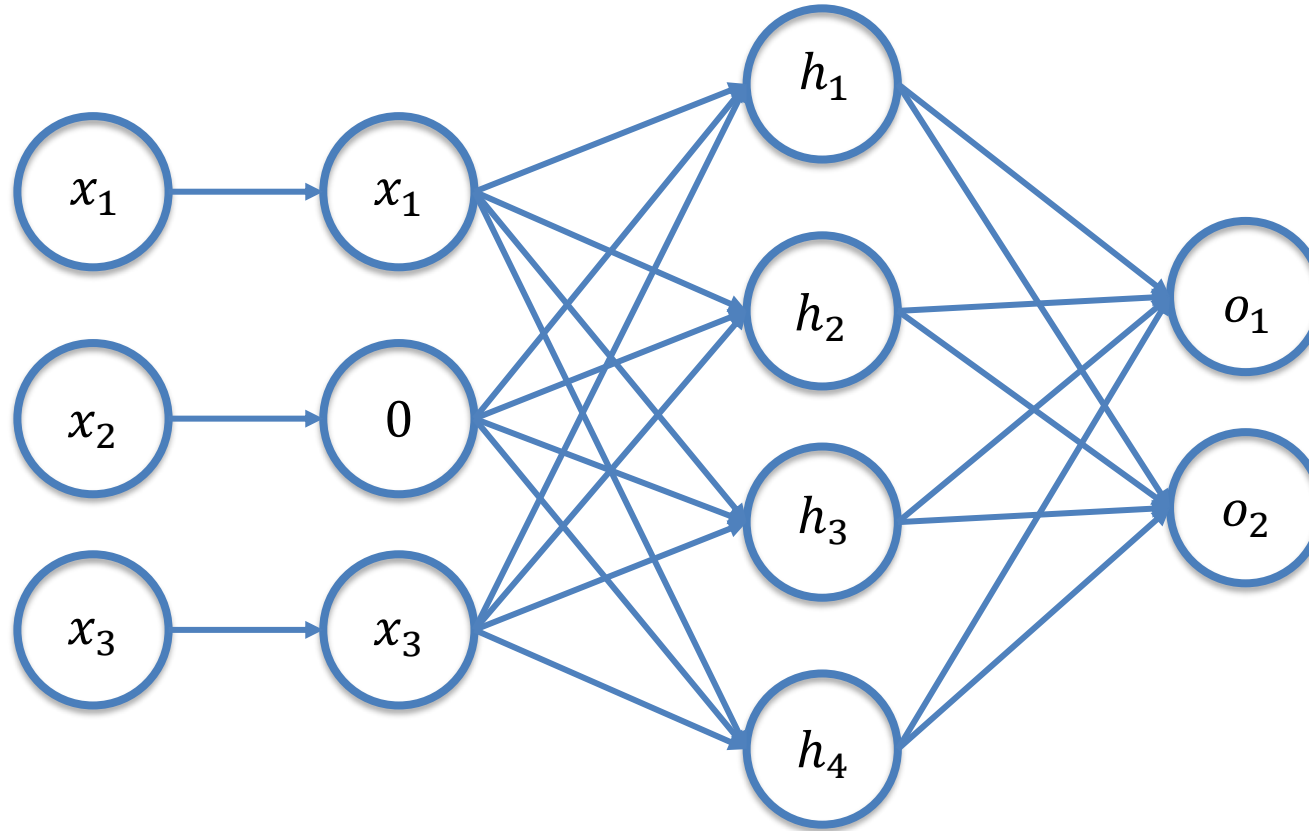
- Training data near the critical amount.
- An overparameterized model.
- Regularization pressure, especially weight decay.
- Patience.

Grokking Modular Addition

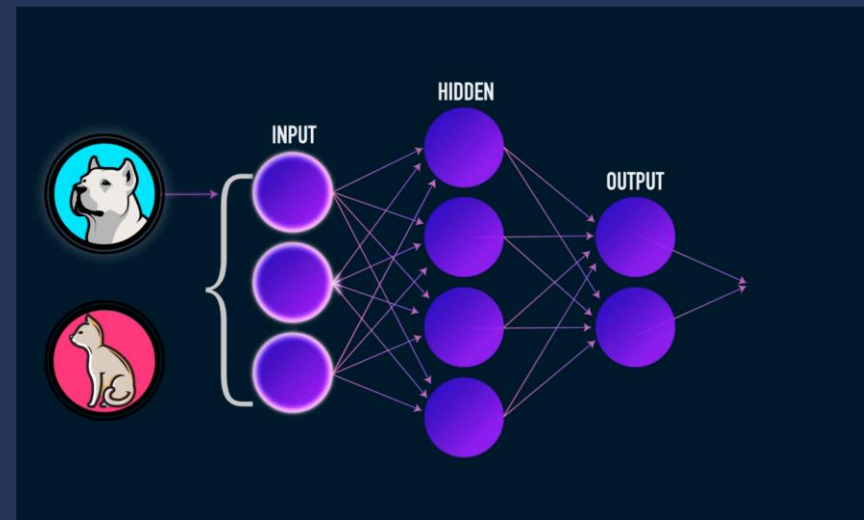


Regularization Strategies: Dropout

- ❖ Zero out randomly selected inputs.



Optimization for Deep Learning



Empirical Risk Minimization

- ❖ Goal of a machine learning algorithm is to reduce the expected generalization error also known as risk: $J^*(\theta) = \mathbb{E}_{(x,y) \sim p_{data}} L(f(x; \theta), y)$
- ❖ p_{data} is the true distribution which is not known, and we only have a training set during model development.
- ❖ The p_{data} is replaced with empirical distribution and we minimize empirical risk.

Batch and Minibatch Algorithms

- ❖ **Batch or deterministic** gradient methods: Use the entire training set.
- ❖ **Stochastic or online** gradient descent: Use one example at a time.
- ❖ **Minibatch *stochastic* methods**: More than one but fewer than all.

Basic Algorithms: Stochastic Gradient Descent

Algorithm 8.1 Stochastic gradient descent (SGD) update

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

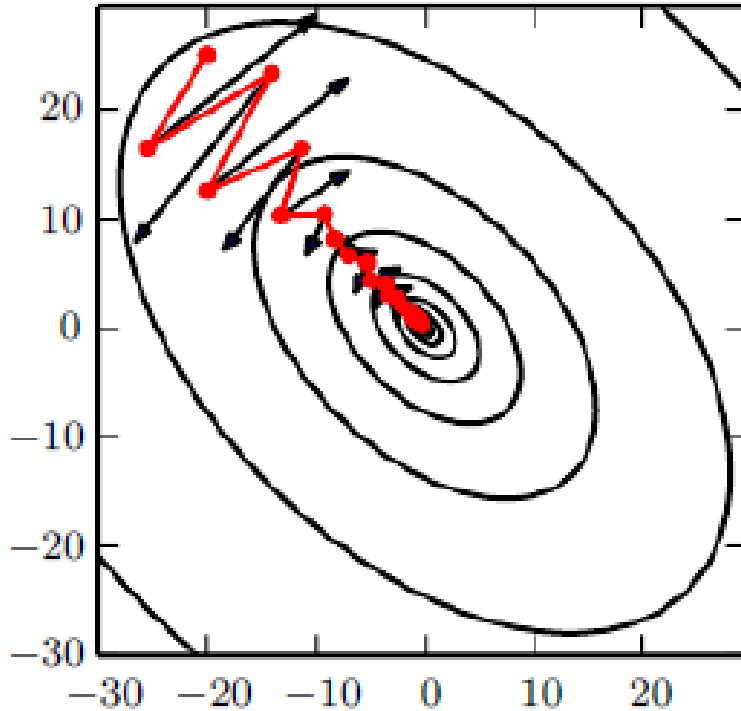
$k \leftarrow k + 1$

end while

[Image Source](#)

Basic Algorithms: Momentum

Without Momentum



[Image Source](#)

- ❖ Direction of the update has some variance.
- ❖ **With Momentum**, the updates are smoother.
 - Stores the directions of previous gradients, v .
 - **Moves** faster if the directions are similar.

Basic Algorithms: Momentum

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$.

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$.

 Apply update: $\theta \leftarrow \theta + \mathbf{v}$.

end while

[Image Source](#)

Hyperparameters

- Depth
- Width
- Activation
- Learning rate
- Batch size
- Epochs
- Dropout rate
- Weight decay

Parameter Initialization

- ❖ **Speeds** up convergence of gradient descent.
- ❖ **Can** lead to a lower generalization error.
- ❖ Different Strategies:
 - Zero Initialization: Not effective
 - Random Initialization: breaks symmetry, large values are problematic.
 - He Initialization: random initialization * $\sqrt{\frac{2}{\text{dimension of the previous layer}}}$

Questions to consider

❖ What happens if the batch size is too small? Too large?

❖ Too small:

- See new instances in every step
- Average loss very noisy

❖ Too large:

- Expensive

Questions to consider

❖ What happens if the learning rate is too small? Too large?

❖ Too small:

- Minor updates to parameters in each iteration
- Training time is very high

❖ Too large:

- Noisy updates and unstable



UNIVERSITY OF
TORONTO

Engineering

Thank you.